

Architecture and Implementation Plan for a Scalable Transaction Data Validation Platform

1. Problem Understanding and Requirements

The goal is to build a web-based platform for transaction data validation and processing that accepts CSV datasets containing order-level details, product-level details, and payment mode information and then performs comprehensive validation before emitting a cleaned, downloadable output and splitting very large CSVs into smaller chunks.

The platform must support phone number validation with country-specific rules, robust date/time validation, and general data integrity checks while remaining scalable, user-friendly, and able to handle diverse international formats.^{[1][2]}

1.1 Functional Requirements (Inferred from Prompt)

- Upload transaction datasets in CSV (and optionally TSV/Excel) containing multiple logical tables (orders, products, payments) either in a single file or as multiple files.
- Validate each row and column according to configurable rules:
 - Phone numbers validated against country-specific rules (e.g., Singapore 8 digits, India 10 digits) with support for different formats, prefixes, and country codes.^{[3][4][^5]}
 - Date and time validation against predefined formats and logical constraints (e.g., order date before delivery date, timezone awareness).
 - General data integrity checks: required fields, type checks, range checks, referential integrity between order/product/payment records.
- Generate a cleaned and validated output file, e.g., a new CSV or parquet with invalid records flagged or separated.
- Automatically split large output files into smaller manageable chunks for easier download and further processing.^{[6][7]}
- Provide a web UI for file upload, configuration of validation rules (or selection of predefined templates), and download of results.
- Expose APIs so the platform can be integrated into other systems and automated pipelines.

1.2 Non-Functional Requirements

- **Scalability:** Ability to process large CSV files (tens of GB) and many concurrent uploads without exhausting memory or blocking the web tier.^{[2][8][^9]}
- **Reliability:** Resilient to network failures during upload (support resumable/chunked uploads), with clear error reporting and logging.^{[10][11][^12]}
- **Performance:** Efficient parsing and validation using streaming and vectorized operations where possible (e.g., Polars or similar columnar engine).^{[8][9][^2]}

- **Extensibility:** Modular validation rule engine that can be extended with new rules without redesigning the system.
- **Security:** Authentication for upload/download, input sanitization, size limits, anti-abuse controls.

2. High-Level System Architecture

The system is best implemented as a multi-tier web application with a separate compute layer for heavy CSV processing and object storage for large files.

This avoids routing all data through a single synchronous web request and enables scaling of processing workers independently of the UI/API tier.^{[11][6][^7]}

2.1 Core Components

- **Web Frontend:** React/Next.js or similar SPA for upload UI, progress tracking, and result downloads.
- **API Gateway / Backend:** FastAPI or Node.js (NestJS/Express) service that exposes REST endpoints for:
 - Upload session creation and presigned URL generation for direct-to-object-storage uploads.
 - Validation job creation and status polling.
 - Download URLs for cleaned outputs and chunked files.
- **Object Storage:** AWS S3 / GCP Cloud Storage / Azure Blob for raw uploads and final outputs; enables direct client uploads using presigned URLs and scalable storage.^{[11][7]}
- **Job Queue:** Message queue like AWS SQS, RabbitMQ, or Kafka to decouple upload completion from processing and support retry semantics.
- **Worker Service (Data Processing Engine):** Python service using Polars for CSV parsing/validation and phonenumbers library for phone validation.^{[2][8][3][9][^5]}
- **Metadata Database:** Relational DB (PostgreSQL) to track files, jobs, validation configurations, and status.
- **Monitoring and Logging:** Centralized logs (e.g., ELK stack), metrics (Prometheus/Grafana), and alerting.

2.2 Data Flow Overview

1. User starts an upload from the web UI; frontend requests an upload session from the backend.
2. Backend creates a file metadata record in the database and generates a presigned upload URL from object storage.^{[11][7]}
3. Frontend uploads the file in chunks directly to object storage using the presigned URL with client-side chunking for resumable uploads.^{[10][7][^11]}
4. On completion, the backend enqueues a validation job in the message queue referencing the stored file.

5. Worker pulls the job, streams the CSV from storage, validates it using Polars + rule engine, writes cleaned data and error reports to object storage in chunked outputs, and updates job status.^{[2][8][^9]}
6. User polls job status via UI or gets notified (e.g., email or webhooks); once complete, they can download cleaned files and view validation reports.

3. Recommended Tech Stack

3.1 Frontend

- **Framework:** Next.js (React) with TypeScript for SSR/SSG where useful and SPA behaviour for upload flows.
- **UI Library:** Tailwind CSS or Chakra UI for rapid layout and React Query (TanStack Query) for state management of async job polling.
- **Upload Handling:** Use browser File API for client-side chunking; integrate with presigned URLs or Uploadcare/Resumable.js style libraries to support resumable, chunked uploads.^{[11][7]}

3.2 Backend/API

- **Framework:** FastAPI (Python), leveraging async endpoints for lightweight API operations and reuse of Python validation code for smaller synchronous datasets.
- **Authentication:** JWT-based auth (e.g., using OAuth2 with JWT in FastAPI) and API keys for system-to-system integration.
- **Object Storage Integration:** boto3 (AWS), google-cloud-storage, or Azure SDK depending on cloud provider; presigned URL generation for direct uploads.^{[11][7]}
- **Background Tasks:** Celery or RQ with Redis, or AWS SQS/Lambda for serverless-style processing.

3.3 Data Processing Engine

- **Language:** Python for its rich data stack and phonenumbers library.^{[1][3][4][5]}
- **CSV Engine:** Polars for high-performance CSV reading and processing in streaming/lazy mode to handle larger-than-memory datasets and efficient column selection.^{[2][8][^9]}
- **Phone Validation:** phonenumbers (port of Google libphonenumber) for parsing, validating, and formatting international numbers according to country-specific rules.^{[3][4][5][1]}
- **Date/Time Parsing:** Python datetime, dateutil, or pendulum for parsing multiple formats and timezones.
- **Schema/Rule Definitions:** Pydantic models or declarative YAML/JSON for validation rules per dataset type.

3.4 Storage and Infrastructure

- **Relational DB:** PostgreSQL for metadata, job tracking, and configuration tables.
- **Object Storage:** AWS S3 buckets organised by tenant/project and file type (raw, intermediate, output).
- **Deployment:** Containerized services (Docker) orchestrated via Kubernetes (EKS/GKE/AKS) or simpler ECS/Fargate setup for initial deployment.
- **CI/CD:** GitHub Actions or GitLab CI for test, build, and deploy pipelines.

4. Detailed Validation Design

4.1 Schema and Rule Engine

Define schemas and rules per dataset type (orders, products, payments) in a declarative format that the worker engine interprets at runtime.

This allows new datasets or countries to be supported by adding configuration instead of changing core code.

Example structure (in JSON/YAML terms):

- **fields:** list of column definitions with:
 - **name** (e.g., `order_id`, `order_date`, `customer_phone`)
 - **type** (string, integer, float, date, datetime, enum)
 - **required** flag
 - **constraints** (regex, min, max, allowed values)
- **relations:** referential integrity constraints (e.g., `order_id` must exist in the orders table when validating payments).
- **country_rules:** phone length, required prefixes, default country code inferred from country column.

Validation operations:

- **Column-level checks:** Type casting, null/empty checks, length, regex match, allowed value sets.
- **Cross-field checks:** e.g., `ship_date ≥ order_date`, currency and amount logic.
- **Cross-table checks:** Ensure foreign keys exist and match constraints.

4.2 Phone Number Validation Strategy

- Use `phonenumbers.parse` with explicit region derived from a `country_code` or `country_iso` column, or fallback default when missing.^{[3][4][^5]}
- Apply `phonenumbers.is_valid_number` and `is_possible_number` to ensure format and length validity.^{[4][3]}

- Use `phonenumbers.format_number` to normalize into E.164 or national format for consistent downstream processing.^{[1][5][3][4]}
- Optionally use geocoder and carrier functions to enrich validation reports with location/provider for debugging.^[^4]

Implementation pattern in the worker:

- Extract phone columns as a Polars Series, then apply a vectorized map/apply function that calls small compiled functions (possibly via `pyo3` or `numba` wrappers if needed for performance), caching parse results per country/number pattern.

4.3 Date and Time Validation

- Support multiple input formats by attempting parsing with known patterns (e.g., "%Y-%m-%d", "%d/%m/%Y") configured per country or dataset.
- Enforce logical constraints: order date not in the future beyond a tolerance, delivery date after order date, no negative time intervals.
- Store normalized datetimes in ISO 8601 in the output.

4.4 General Data Integrity Checks

- **Required Fields:** Null checks.
- **Type Coercion:** Strict casting with errors captured into a separate error report.
- **Range and Domain Checks:** e.g., amounts ≥ 0 , quantity integer and within reasonable bounds.
- **Uniqueness Constraints:** e.g., `order_id` unique per file or globally per tenant.

4.5 Error Reporting

Design a structured error report per file:

- CSV/Parquet with columns: `row_number`, `field_name`, `error_code`, `error_message`, optional `proposed_fix`.
- Aggregate statistics for UI: counts of errors by type, percentage of rows valid.

The user can choose to:

- Download only valid rows.
- Download all rows with additional boolean flags/columns like `is_valid` and `error_summary`.

5. Handling Large CSVs and Chunking

5.1 Ingestion and Parsing Strategy

For large files (e.g., multiple GB), reading entire CSVs into memory is infeasible.

Polars supports lazy CSV scanning (`scan_csv`) and streaming execution, which allows processing rows in chunks and applying filters/aggregations without loading the full dataset into memory.^{[2][8][^9]}

Key techniques:

- Use `pl.scan_csv` instead of `pl.read_csv` to create a `LazyFrame` with schema inference or explicit schema, enabling pushdown of projections/filters.
- Select only required columns before evaluation so unnecessary columns are never loaded.^[^8]
- Evaluate queries with `collect(streaming=True)` to process data in batches in a streaming fashion, allowing handling of datasets much larger than available RAM.^{[9][8]}

5.2 Output Chunking

Post-validation, the system must split large outputs into smaller chunks.

Strategies include:

- Using Polars streaming to write output in chunks (e.g., 100 MB or N rows per file) as separate CSVs such as `output_part_0001.csv`, `output_part_0002.csv`.
- Tracking generated chunk files in metadata DB for easy listing and download.
- For extremely large results, offering compressed archives (zip) and/or parquet for better compression.

For downloading, streaming APIs can be used to deliver large files, using HTTP chunked transfer encoding instead of constructing the entire response in memory.^[^6]

5.3 Chunked and Resumable Uploads

Client-side chunking and resumable uploads improve reliability for large files.

Best practices from scalable file upload system designs include:^{[10][11][12][7]}

- Split files into fixed-size chunks (e.g., 5–10 MB) and upload each chunk independently, which allows retrying only failed chunks.^{[11][12][7][10]}
- Store metadata for each upload session: file name, total size, chunk count, uploaded chunk indices, checksum, etc.^{[12][11]}
- Use presigned URLs for each chunk so that uploads go directly from client to cloud object storage without traversing the backend.^{[7][11]}
- Provide an endpoint to query upload status and resume from the last successful chunk.^{[11][12]}

6. API Design

6.1 Core Endpoints

- `POST /api/v1/upload-sessions` – Create an upload session, returning upload configuration and presigned URLs (or a token used to request chunk URLs).
- `GET /api/v1/upload-sessions/{id}` – Get session status and metadata.
- `POST /api/v1/validation-jobs` – Trigger validation for a completed upload, specifying dataset type, rule set, and options.
- `GET /api/v1/validation-jobs/{id}` – Poll job status and retrieve links to output and error reports.
- `GET /api/v1/files/{id}` – Get information about generated output files and chunks.

6.2 Request/Response Patterns

- Use JSON for control plane (job creation, status) and HTTP redirects/presigned URLs for data plane (actual file transfers).
- Return job states like `PENDING`, `RUNNING`, `SUCCEEDED`, `FAILED` with progress percentage when feasible.
- Consider websockets or Server-Sent Events for real-time updates when the user waits in the UI.^[^6]

7. Scalability and Reliability Considerations

7.1 Horizontal Scaling

- Run multiple instances of API servers and workers behind a load balancer.
- Scale worker replicas based on queue depth; container orchestration (Kubernetes) can manage autoscaling policies.

7.2 Fault Tolerance

- Ensure idempotency of processing jobs for the same file; use deterministic job IDs based on file hash and dataset type.
- Implement retry with backoff in the worker when reading from storage or writing outputs.
- Use transactional updates in the metadata DB to maintain consistent job state.

7.3 Monitoring and Observability

- Metrics: number of uploads, average processing time per GB, error counts per rule type.
- Tracing: request IDs propagated from frontend through backend and worker to correlate logs.

7.4 Security

- Limit maximum file size (e.g., per tenant) and enforce content-type checks, optionally verify file signatures (magic numbers) to prevent malicious uploads.^[12]
- Short-lived presigned URLs, role-based access to outputs.
- Encrypt data at rest (server-side encryption) and enforce HTTPS everywhere.

8. Dev Workflow and Testing Strategy

8.1 Development Workflow

- Separate repositories or monorepo with distinct frontend, backend, and worker modules.
- Use Docker for local development, with docker-compose to spin up DB, object storage emulator (e.g., MinIO), queue, and services.

8.2 Testing

- **Unit Tests:** Validation rule engine, schema parsing, phone validation utilities (including edge cases per country).^[14]
- **Integration Tests:** End-to-end file processing using sample datasets; tests should verify chunked outputs and correctness of validation reports.
- **Load Tests:** Synthetic large files to test performance characteristics; ensure Polars streaming mode is used and memory bounds are respected.^{[2][8][9]}

9. Incremental Implementation Plan

Phase 1: Minimal Viable Product (MVP)

- Build basic web UI with single-file upload (no chunked uploads yet) targeting moderate file sizes (e.g., ≤ 100 MB).
- Implement synchronous backend endpoint that accepts file upload, runs validation with Polars and phonenumbers in a single process, and returns cleaned CSV plus error report.
- Support limited set of rules: required fields, type checks, simple phone validation for a couple of countries, basic date validation.

Phase 2: Asynchronous Jobs and Object Storage

- Introduce object storage, presigned URL uploads, and message queue.
- Refactor validation into worker service; API only orchestrates jobs and status.
- Add streaming Polars processing and output chunking for large files.

Phase 3: Advanced Validation and Internationalization

- Extend rule engine to multiple countries, complex cross-field relations, and configurable rule sets per tenant.
- Add aggregated error dashboards and metrics.

Phase 4: Hardening and Production Readiness

- Implement auth, rate limiting, audit logging, and full observability.
- Optimize performance, conduct load tests, and tune worker autoscaling.

10. Trade-offs and Alternatives

- **Polars vs Pandas:** Polars provides superior performance, multi-threading, and streaming for large CSVs compared to Pandas, which typically requires loading entire datasets into memory.^{[2][8][^9]}
- **Monolith vs Microservices:** A modular monolith might be simpler initially, but separating worker and API services early offers clearer scaling and isolation of CPU-intensive workloads.
- **Direct Upload to Backend vs Object Storage:** Direct-to-storage with presigned URLs reduces backend load and improves scalability compared to proxying uploads through the API.^{[11][7]}
- **Custom Upload Implementation vs Managed Service:** Using libraries/platforms like Uploadcare can accelerate development but introduces vendor lock-in and ongoing costs; building a custom chunked uploader provides more control but more engineering effort.^[^7]

This architecture and implementation plan provide a concrete blueprint for an AI-empowered transaction data validation platform that is scalable, robust, and extensible, with clear layering between frontend, APIs, and data processing workers. The design emphasizes streaming data processing, chunked uploads, and a modular validation engine, making it well suited as a foundation for further AI-driven enhancements such as anomaly detection or auto-suggested corrections in future iterations.^{[8][9][6][2][11][7]}

References

1. [Phonenumbers module in Python](#) - phonenumbers is one of the modules that provides numerous features like providing basic information ...
2. [Reading CSV files in Polars](#). - Polars is faster than Pandas in many ways. One particular way includes reading in a csv file.
3. [Phonenumbers Module in Python](#) - The phonenumbers module in Python simplifies parsing, formatting, and validation of phone numbers. B...
4. [phonenumbers library in python](#) - The library in python named "phonenumbers" and with this library I easily extract the certain inform...

5. [phonenumbers](#) - Python version of Google's common library for parsing, formatting, storing and validating internatio...
6. [Design architecture for large file downloads](#) - You are tasked with implementing a feature that allows users to download a massive CSV file, weighin...
7. [How to upload large files: Developer guide](#) - 1. Chunk file uploads: Split large files into manageable parts · 2. Resumable uploads: Ensure reliab...
8. [To go big you must be lazy](#) - This means that Polars takes care of scanning all the CSV files and concatenating them into a single...
9. [Working with larger-than-memory datasets with Polars](#) - Polars makes it easy to work with very large datasets even on a regular laptop. In this video I show...
10. [Chunked Uploads - Box Dev Docs](#) - The Chunked Upload API provides a way to reliably upload large files to Box by chunking them into a ...
11. [Designing a Scalable File Upload System: Best Practices](#) - Each chunk is uploaded independently, Avoid routing large files through your backend. Instead, use p...
12. [Building a Scalable File Upload Service with Go: Complete ...](#) - This comprehensive guide explores how to implement a production-ready file upload system using Go (G...